

AFRILANG Stdlib

std/c/parsetemplate

Français. Bibliothèque AFRILANG 'std/c/parsetemplate' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : tempLen, tempUpper, tempLower, tempTrim, tempSplit, tempJoin, tempReplace.

'import "std/c/parsetemplate"' · 'use parsetemplate'

- 'tempLen(s text) → number' - 'tempUpper(s text) → text' - 'tempLower(s text) → text' - 'tempTrim(s text) → text' - 'tempSplit(s text, delim text) → list text' - 'tempJoin(parts list text, delim text) → text' - 'tempReplace(s text, from text, to text) → text'

English. AFRILANG library 'std/c/parsetemplate' (tier complex, category complex). Exports 7 function(s): tempLen, tempUpper, tempLower, tempTrim, tempSplit, tempJoin, tempReplace.

'import "std/c/parsetemplate"' · 'use parsetemplate'

- 'tempLen(s text) → number' - 'tempUpper(s text) → text' - 'tempLower(s text) → text' - 'tempTrim(s text) → text' - 'tempSplit(s text, delim text) → list text' - 'tempJoin(parts list text, delim text) → text' - 'tempReplace(s text, from text, to text) → text'

Catégorie / Category	Modules complexe (c/)
Niveau / Tier	complexe
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/parsetemplate' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : tempLen, tempUpper, tempLower, tempTrim, tempSplit, tempJoin, tempReplace.

'import "std/c/parsetemplate"' · 'use parsetemplate'

```
- `tempLen(s text) → number`
- `tempUpper(s text) → text`
- `tempLower(s text) → text`
- `tempTrim(s text) → text`
- `tempSplit(s text, delim text) → list text`
- `tempJoin(parts list text, delim text) → text`
- `tempReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parsetemplate"
use parsetemplate
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- tempLen(s text) → number•
tempUpper(s text) → text•
tempLower(s text) → text•
tempTrim(s text) → text•
tempSplit(s text, delim text) → list•
tempJoin(parts list text, delim text) → text•
tempReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/parsetemplate"
use parsetemplate
```

```
create result = tempLen(/* args */)
say result
```

```
create result = tempUpper(/* args */)
say result
```

```
create result = tempLower(/* args */)
say result
```

```
create result = tempTrim(/* args */)
say result
```

```
create result = tempSplit(/* args */)
say result
```

```
create result = tempJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/parsetemplate"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module parsetemplate

```
function tempLen(s text) returns number
end
```

```
function tempUpper(s text) returns text
end
```

```
function tempLower(s text) returns text
end
```

```
function tempTrim(s text) returns text
end
```

```
function tempSplit(s text, delim text) returns list text
end
```

```
function tempJoin(parts list text, delim text) returns text
end
```

```
function tempReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/parsetemplate' (tier complex, category complex). Exports 7 function(s): tempLen, tempUpper, tempLower, tempTrim, tempSplit, tempJoin, tempReplace.

'import "std/c/parsetemplate"' · 'use parsetemplate'

```
- `tempLen(s text) → number`
- `tempUpper(s text) → text`
- `tempLower(s text) → text`
- `tempTrim(s text) → text`
- `tempSplit(s text, delim text) → list text`
- `tempJoin(parts list text, delim text) → text`
- `tempReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parsetemplate"
use parsetemplate
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- tempLen(s text) → number•
tempUpper(s text) → text•
tempLower(s text) → text•
tempTrim(s text) → text•
tempSplit(s text, delim text) → list•
tempJoin(parts list text, delim text) → text•
tempReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/parsetemplate"
use parsetemplate
```

```
create result = tempLen(/* args */)
say result
```

```
create result = tempUpper(/* args */)
say result
```

```
create result = tempLower(/* args */)
say result
```

```
create result = tempTrim(/* args */)
say result
```

```
create result = tempSplit(/* args */)
say result
```

```
create result = tempJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/parsetemplate"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module parsetemplate

```
function tempLen(s text) returns number
end
```

```
function tempUpper(s text) returns text
end
```

```
function tempLower(s text) returns text
end
```

```
function tempTrim(s text) returns text
end
```

```
function tempSplit(s text, delim text) returns list text
end
```

```
function tempJoin(parts list text, delim text) returns text
end
```

```
function tempReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/parsetemplate"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/parsetemplate/>

Source (excerpt)

```
module parsetemplate

function tempLen(s text) returns number
end

function tempUpper(s text) returns text
end

function tempLower(s text) returns text
end

function tempTrim(s text) returns text
end

function tempSplit(s text, delim text) returns list text
end

function tempJoin(parts list text, delim text) returns text
end

function tempReplace(s text, from text, to text) returns text
end
end
```